

UI STM 구현 설명서

2026-09-17 개정: STM-Controller 책임 분리 이후의 실제 구현을 본문과 코드 예시에 반영했습니다. 파일명의 날짜는 최초 작성일입니다. 변경 전후 비교와 상세 검증 기록은 [UI STM-Controller 책임 분리 설명서](#)에서 확인할 수 있습니다.

기준: 2026-09-17 작업 폴더의 실제 구현 · XState 5.32.5

설계 문서: [UI_Event_Action_Table.md](#) · [UI_Rule.md](#) · [UI_Behavior.md](#)

교수님, 이 프로젝트의 UI는 **UISTM**이 하나의 **XState** 루트 상태 계층을 순수하게 평가하고, **UIStateController**가 그 결과에 따라 외부 작업을 실행하는 구조입니다. 상단 상태바, 화면 선택, 프로그램 종료가 동시에 활성화되고, 선택된 화면 안에서 REGIME·차트·계좌·거래 내역 같은 하위 Region이 각자의 상태를 유지합니다. 기존의 상태 계층과 동작은 유지하면서 판단과 실행의 소유자를 분리했습니다.

이 문서는 설계의 32개 Event-action table, 총 182개 행을 실제 파일과 실행 경로에 연결합니다. 상태를 판단하는 코드, 명령을 실행하는 코드, 화면을 그리는 코드를 구분하고, 마지막에는 실제 코드 예시와 전체 ID 색인을 제공합니다.

1. 전체 구조와 책임

교수님, 화면의 버튼을 누른다고 React가 곧바로 Python의 **TradingController**를 호출하는 것은 아닙니다. 다음 경계를 통과합니다.

사용자 조작

- React 컴포넌트의 callback
- `UiApplicationStore.dispatch(intent)`
- `UIStateController.dispatch(intent)`
- `UiIntentRouter`: 입력을 내부 이벤트 묶음으로 변환
- Controller 직렬 큐: 이벤트와 시간 입력 준비
- **UISTM**: 현재 상태·event·guard에 따라 순수 전이 평가
- `UITransitionResult { snapshot, actions }`
- Controller: 최종 snapshot 반영 → Action 순서대로 실행 → 구독자 발행

└─ 화면 모델 → React 렌더링

└─ `UiCommandExecutor` → `UiCommandPort` / 타이머

↓

`BackendUiAdapter` / 네이티브 기능

↓

HTTP backend / 폴더 선택 / CSV / 종료

↓

완료·실패·만료 이벤트 → Controller 직렬 큐 → **UISTM**

서버가 먼저 알려 주는 계좌·체결·전략 상태도 **BackendUiAdapter**와 **backendEventMapper**를 거쳐 같은 Controller 큐로 들어옵니다. 기존 이름인 **UiApplicationFacade**는 **UIStateController**를 그대로 재수출하는 호환 진입점입니다. 두 이름이 같은 클래스를 가리키며 별도 상태나 실행 경로를 만들지 않습니다.

구성 요소	판단하거나 실행하는 내용
UISTM	상태 전이, 내부 context 변경, 요청 token·최신 결과 수락, 필요한 Action 요청 결정
UIStateController	입력 변환, STM 호출, Action 실행 지시, 결과 이벤트 전달, 구독·화면 모델·실행 수명 관리
Controller 소유 UiCommandExecutor	Promise·AbortController·타이머 핸들 관리와 실제 Port 호출
React와 Store	snapshot에서 계산한 표시값을 구독하고 화면을 렌더링

STM은 API·폴더 선택·CSV·타이머·종료를 직접 실행하지 않습니다. Controller와 실행부는 STM의 전이 조건을 다시 판단하지 않고 반환된 요청을 실행합니다. 이 경계는 UI 상태 머신의 작업에 대한 경계이며, 차트 데이터 구독·프레임 렌더링 같은 기존 표시 전용 수명까지 이번에 옮긴 것은 아닙니다.

여기서 UI STM이 책임지는 것은 확인창, 처리 중 표시, 필터, 탭, 차트 조작, 결과 표시와 종료 절차입니다. 실제 거래 전략 판단과 주문·체결 반영은 backend가 책임집니다. UI의 **running**은 자동매매 실행 상태를 나타내며, 개별 매수 주문이 체결됐다는 뜻은 아닙니다.

2. 어떤 폴더와 파일을 읽으면 되는가

2.1 애플리케이션 공통 실행부

파일	실제 역할
app/machines/UISTM.ts	루트 snapshot 보관, initialTransition·transition 평가, 순서 있는 Action 요청 반환
app/machines/uiApplicationMachine.ts	최상위 상태, 실제 Region 포함 관계, 내부 이벤트 묶음, 화면 history, 루트 final 구성
app/machines/uiFeatureDefinitions.ts	외부 함수가 없는 초기 데이터로 기능별 상태·가드·내부 Action·작업 요청 정의 준비
app/machines/uiRegionComposition.ts	기능 정의를 루트 상태 노드로 조립하고 root context·이벤트·명령 결과에 연결
app/machines/uiApplicationTypes.ts	공통 context, 요청 token, 내부 event, 루트 snapshot 계약
app/machines/uiActions.ts	UIActionRequest·UITransitionResult ·시간 입력의 데이터 계약
app/control/UIStateController.ts	STM 생성·호출, 입력/결과 직렬 큐, 시간 공급, 작업 실행·구독·시작·종료
app/control/UiCommandExecutor.ts	Controller 소유 실행부; Port 호출, 작업 교체·취소, 타이머, 완료·실패 전달
app/control/uiApplicationFacade.ts	UIStateController 의 호환 이름과 기존 타입·selector 재수출
app/control/uiApplicationIntents.ts	화면별 입력 허용, intent 변환, 서버 데이터 동기화 이벤트 구성

파일	실제 역할
app/control/uiApplicationContracts.ts	UiApplicationIntent · AppViewModel ·초기 옵션 계약
app/machines/uiApplicationViews.ts	루트 snapshot에서 기능별 읽기 전용 view 추출
app/control/selectAppViewModel.ts	React가 사용할 표시값과 pending·오류·모달 상태 계산
app/control/uiModalPolicy.ts	현재 상태에서부터 표시할 모달 하나를 결정
app/runtime/UiApplicationStore.ts	화면 모델 캐시, React 구독, 알림 시점 관리
app/bootstrap/createLiveUiApplication.ts	backend 초기 snapshot, Controller, 실시간 이벤트 연결; 기존 Facade import 사용
shared/ports/UiCommandPort.ts	UI가 요청할 수 있는 명령의 인터페이스
shared/api/BackendUiAdapter.ts	실제 HTTP·응답 검증·버전·연결·종료 조정
shared/api/backendEventMapper.ts	backend 데이터를 UI 표시 계약과 intent로 변환

2.2 기능별 정의

아래 파일들은 모두 [UI/src/features/](#) 아래에 있습니다. [*Machine.ts](#)는 순수 상태·가드·내부 Action·작업 요청 정의를 제공하고, [*Regions.ts](#)는 루트에 포함할 상태 계층을 구성합니다. 기능별 정의는 Port나 Promise actor를 갖지 않습니다.

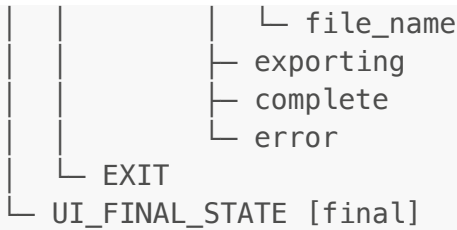
기능 폴더	실행 정의	담당 표
connection-status/machines	connectionMachine.ts	U1
trading-control/machines	tradingCommandMachine.ts , tradingRegions.ts	U2·U3
regime-selection/machines	regimeMachine.ts , regimeRegions.ts	R1·R2·R3
price-chart/machines	chartMachine.ts	DC1~DC6·IP1~IP3
account-summary/machines	accountSummaryMachine.ts	DI1·DI2
split-order/machines	splitOrderMachine.ts , splitOrderRegions.ts	SI·SO
recent-orders/machines	recentOrdersMachine.ts	M4
trade-history/machines	tradeHistorySummaryMachine.ts , tradeHistoryMachine.ts , historyFilterRegions.ts	D1~D4·TD2·TD3
csv-export/machines	csvExportMachine.ts , csvExportRegions.ts	TD4·CR1~CR3
app-exit/machines	appExitMachine.ts	ES3

화면 이동 ES2는 `uiApplicationMachine.ts`에 있습니다. 각 기능 폴더의 `components/`와 `App.tsx·AppModalHost.tsx`는 화면 모델을 받아 실제 화면과 입력 callback을 구성합니다.

3. 실제 XState 상태 계층

다음 트리는 코드의 상태 이름을 사용합니다. `parallel` 아래의 Region들은 부모가 활성화된 동안 함께 활성화됩니다. `SCREEN`처럼 `parallel`이 아닌 복합 상태에서는 자식 중 하나만 활성화됩니다.

```
uiApplicationMachine
├─ ETIRE_UI_SYSTEM [parallel]
│   └─ UPPER_STATUS_BAR [parallel]
│       ├── API_DISPLAY
│       ├── STOP_BUTTON
│       └─ START_BUTTON
├─ SCREEN [하나의 화면 선택]
│   ├── MAIN_SCREEN_WRAPPER [parallel]
│   │   ├── HISTORY [deep history pseudo state]
│   │   ├── REGIME_PANEL [parallel]
│   │   │   ├── recommendation
│   │   │   ├── selection
│   │   │   └─ indicators
│   │   ├── DISPLAY_CHART [parallel]
│   │   │   ├── interval
│   │   │   ├── indicator_settings
│   │   │   │   └─ opened [parallel: bollinger_bands / ema9 / volume]
│   │   │   ├── active_state
│   │   │   ├── viewport
│   │   │   ├── drawing
│   │   │   └─ line_selection
│   │   ├── DISPLAY_ACCOUNT_INFO [parallel]
│   │   │   ├── trading_logic_status
│   │   │   ├── asset_summary
│   │   │   └─ SPLIT_ORDER [parallel]
│   │   │       ├── scale_in → SCALE_IN_ORDER → ready / saving / failed
│   │   │       └─ scale_out → SCALE_OUT_ORDER → ready / saving / failed
│   │   └─ TRADER_PANEL
│   └─ TRADING_DETAILS [parallel]
│       ├── HISTORY [deep history pseudo state]
│       ├── ACCOUNT_DETAILS [parallel]
│       │   ├── profit_rate
│       │   ├── sell_performance
│       │   ├── eth_holdings
│       │   └─ daily_trading_fee
│       ├── PERIOD [parallel]
│       │   ├── selection → today / last7days / last30days / all
│       │   └─ query → idle / loading / ready / empty / failed
│       ├── SIDE → all / buy / sell
│       └─ CSV_EXPORT
│           ├── closed
│           ├── editing [parallel]
│           │   └─ file_browser
│           └─ period
```



HISTORY는 복귀 경로를 기억하는 의사 상태입니다. 메인의 다섯 번째 실행 Region이나 상세 화면의 다섯 번째 실행 Region으로 세지 않습니다. 메인에는 네 개, 상세에도 네 개의 실행 Region이 있습니다.

ETIRE_UI_SYSTEM이라는 철자는 코드와 설계 문서에서 사용하는 식별자입니다. 루트 snapshot의 **value**에서 **UPPER_STATUS_BAR·SCREEN·EXIT**를 직접 확인할 수 있습니다. 메인과 상세가 동시에 활성화되지는 않지만, 어느 화면 이든 상단과 종료 Region은 계속 활성화됩니다.

3.1 machine 정의, 순수 전이 평가, 실행 인스턴스

machine은 상태·가드·Action 규칙의 정의입니다. **create_feature_definitions()**가 여러 machine 정의를 만들지만, 이들은 한 루트에 조립됩니다. 현재 UI 실행 경로는 **createActor()**로 루트 actor나 작업용 Promise actor를 시작하지 않습니다.

UISTM의 생성자는 초기 상태와 아직 실행하지 않은 요청을 다음처럼 계산합니다.

```

constructor(private readonly machine: AnyStateMachine) {
  const [snapshot, actions] = initialTransition(machine);
  this.snapshot = snapshot;
  this.initial_actions = this.read_actions(actions);
}
  
```

이후 **handle(event, input)**이 **transition(machine, snapshot, event)**로 다음 상태와 요청을 계산합니다. **assign**과 즉시 발생하는 내부 이벤트는 이 평가 안에서 처리됩니다. 반환한 요청을 실제로 실행하는 주체는 Controller입니다.

UISTM은 최신 snapshot을 보관하므로 클래스 자체는 상태를 갖습니다. 여기서 “순수”는 **동일한 상태·이벤트·시간 입력에 대해 외부 작업 없이 같은 전이 결과를 계산한다**는 뜻입니다. XState의 계층·병렬·history 규칙은 그대로 사용되, 실행 중인 actor가 I/O를 소유하던 경계를 없앴습니다.

3.2 기능 정의를 실제 루트 노드로 연결하는 방법

UiRegionComposition.compile()은 다음 연결을 만듭니다.

1. 기능의 **TYPE_CLICKED**를 **regime.TYPE_CLICKED**처럼 이름이 겹치지 않는 루트 이벤트로 바꿉니다. 원래 event 데이터는 **source**에 담습니다.
2. 기능 가드와 Action에는 루트 전체가 아니라 **context.features.regime** 같은 해당 기능 데이터를 전달합니다.
3. **assign**이 반환한 변경값을 해당 **features** 항목에 반영하면서 다른 기능 데이터는 보존합니다.
4. 전이 target을 루트에서 찾을 수 있는 상태 ID로 연결합니다.
5. **meta.command**의 작업 종류·입력 계산·성공/실패 규칙을 데이터 요청과 token 검사가 있는 결과 전이에 연결합니다.

6. `meta.timers`의 시간 규칙을 절대 만료 시각이 있는 타이머 요청으로 바꿉니다.

이 클래스가 현재 상태를 별도 반복문으로 판정하는 것은 아닙니다. 최종 구성은 XState 상태 노드이고, 이벤트에 반응할 전이와 guard를 선택하는 주체도 XState입니다. 조립된 루트에는 `invoke.after` 실행 actor가 없으며, 조립 과정에서도 Port 호출이나 타이머 생성을 하지 않습니다.

4. Region의 병렬성, 공통 데이터, 이벤트 처리 단위

4.1 병렬은 독립 상태의 동시 활성화를 뜻합니다

교수님, 차트의 주기가 `4h`이면서, 지표 설정창이 열려 있고, EMA는 꺼져 있으며, 그리기 도구는 대기 중일 수 있습니다. 이를 하나의 거대한 상태 이름으로 조합하지 않고 각각의 Region이 표현합니다.

예를 들어 EMA 끄기는 `indicator_settings.opened.ema9`에 영향을 주고, 주기 `interval`이나 BB·거래량의 선택을 바꾸지 않습니다. 그러나 필요한 가드는 다른 조각의 데이터를 참고할 수 있습니다. 선이 선택된 동안 그리기를 시작하지 못하게 하는 `is_line_selection_idle`이 그런 경우입니다.

이 병렬 구조가 Region마다 JavaScript 스레드를 만든다는 뜻은 아닙니다. HTTP·타이머 같은 비동기 작업은 완료를 기다리는 동안 다른 입력과 함께 진행될 수 있지만, UI 상태 전이는 Controller의 직렬 큐 순서에 따라 STM이 평가합니다.

4.2 상태 경로와 데이터의 차이

루트 snapshot에는 다음 정보가 있습니다.

위치	의미
<code>snapshot.value</code>	지금 활성화된 상태와 Region의 실제 계층
<code>context.features</code>	REGIME 후보·확정값, 차트 설정, 계좌, 거래 목록 등 기능별 데이터
<code>context.server_snapshot</code>	마지막으로 수신한 전체 backend snapshot. 개별 통지의 최신 표시값은 <code>features</code> 에 반영
<code>context.requests</code>	명령별 token과 pending·done·error
<code>context.request_sequence</code>	새 작업에 부여할 단조 증가 식별자
<code>context.summary_revision</code>	오래된 조회 결과가 최신 요약을 덮어쓰지 못하게 하는 기준
<code>context.retained / retained_details</code>	화면 밖 기능의 읽기 전용 view를 만들기 위해 보관한 상태값
<code>context.deferred</code>	화면 밖에서 처리한 결과를 복귀 후 상태 전이로 정리할 내부 이벤트
<code>context.evaluation</code>	Controller가 전달한 현재 시각·단조 시각·CSV 날짜와 전이 직전 상태값

가드는 현재 활성 상태에서 해당 전이가 허용되는지 검사하며 event와 context를 사용할 수 있습니다. 예를 들어 API 연결 여부, 적용 REGIME, 날짜 범위, 선택된 선 ID는 단순한 상태 이름만으로 표현하지 않고 context 또는 event로 전달합니다.

START와 STOP Region은 `context.features.trading` 하나를 공유합니다. 두 Region이 각자 별도의 실행 여부나 포지션의 진실을 소유하지 않습니다.

4.3 한 입력 처리와 비동기 완료를 구분해야 합니다

UiIntentRouter는 하나의 외부 intent를 내부 이벤트 여러 개로 바꿀 수 있습니다. Controller는 이를 **ui.batch** 하나로 큐에 넣습니다. STM은 **ui.evaluate**로 평가 입력을 반영한 뒤 **enqueue.raise()**로 내부 이벤트를 처리하고, 가드·내부 Action·이벤트 없는 전이까지 안정된 최종 snapshot과 요청 목록을 반환합니다.

이 과정은 여러 microstep으로 이어질 수 있습니다. 루트는 현재 입력에서 이어진 내부 이벤트와 전이를 처리해 안정된 상태에 도달한 다음, 다음 외부 이벤트를 처리합니다. 병렬 Region도 이 한 루트의 처리 단위 안에서 전이를 선택합니다.

Controller는 **시간 입력 확보 → STM 평가와 최종 snapshot 반영 → 요청 순서대로 시작/취소 → 구독자 발행**의 순서를 지킵니다. Port 호출이나 구독 callback 중 새 입력이 들어와도 현재 처리가 끝난 뒤 큐에서 처리합니다.

따라서 같은 서버 snapshot의 계좌 값만 바뀌고 전략 표시가 아직 바뀌지 않은 중간 상태를 Controller 구독자에게 따로 공개하지 않습니다. 다만 adapter가 **서로 다른 intent**를 여러 번 dispatch하면 각각 별도의 입력 처리입니다. 모든 WebSocket 메시지가 하나의 거대한 원자적 작업으로 합쳐지는 것은 아닙니다.

또한 한 입력의 run-to-completion은 **HTTP 응답까지 전부 기다린다는 뜻이 아닙니다**.

확인 입력 처리

- starting 상태
- 시작 명령 제출
- pending snapshot 공개
- 다른 사용자 입력·서버 통지 처리 가능
- HTTP 완료 이벤트 도착
- running 또는 오류 상태로 전이

비동기 완료는 나중에 들어오는 별도의 이벤트입니다. 중복 확인의 허용 여부와 결과 token의 유효성은 STM의 상태 정의와 가드가 판단합니다. Controller 큐는 Promise 완료를 기다리며 다음 입력을 막지 않습니다.

5. 사용자 입력·서버 통지·화면 갱신

5.1 입력 수락과 명령 성공은 다릅니다

UIStateController.dispatch()의 반환값은 boolean입니다. 기존 **UiApplicationFacade.dispatch()**도 같은 메서드입니다. **true**는 입력 변환 단계에서 수락했다는 뜻이며, HTTP 요청 성공이나 주문 성공을 뜻하지 않습니다. 수락된 이벤트라도 현재 상태에 처리할 전이가 없으면 상태 변화 없이 끝날 수 있습니다.

UiIntentRouter는 현재 화면을 확인합니다. 상세 화면에서 차트 주기를 바꾸는 사용자 intent나 메인 화면에서 CSV를 여는 intent는 거절합니다. 반면 서버 통지와 이미 시작한 작업의 완료는 화면 활성 여부에 관계없이 처리 경로가 있습니다.

5.2 서버 snapshot의 반영

live bootstrap은 최초 snapshot을 가져와 계약을 검사하고 초기 옵션을 구성합니다. 따라서 처음 화면을 그릴 때 무조건 빈 계좌나 미선택 REGIME을 가정하지 않습니다.

BACKEND_SNAPSHOT_SYNCHRONIZED가 들어오면 루트는 먼저 **server_snapshot**을 저장하고, Router가 준비한 계좌·REGIME·계열·분할 설정·매매 상태·상세 요약 동기화 이벤트를 처리합니다. 화면의 차트 설정이나 route처럼 사용자에게 속한 값은 서버 snapshot과 구분하여 유지합니다.

실시간 **ORDER_EXECUTED·PERFORMANCE_UPDATED** 등은 **backendEventManager.ts**에서 UI intent로 변환됩니다. 설계 표의 event 이름과 서버의 wire event 이름이 항상 같을 필요는 없습니다. 중요한 것은 변환 뒤 어느 Region의 데이터와 상태가 갱신되는지입니다.

5.3 화면은 snapshot에서 파생됩니다

`select_app_view_model()`은 현재 루트 상태와 context를 `AppViewModel`로 바꿉니다. `feature_view()`가 만드는 기능별 객체는 읽기 전용 투영이며 별도 actor가 아닙니다.

`UiApplicationStore`는 Controller를 기존 Facade 이름으로 구독하여 최신 화면 모델을 캐시합니다. 서버 통지에 따른 React 알림은 프레임 단위로 합치고, 사용자 dispatch 경계에서는 대기 중인 알림을 즉시 공개합니다. React는 `useUiApplication.ts`의 `useSyncExternalStore` 연결을 통해 화면 모델을 읽습니다.

`derive_active_modal()`은 대체로 **종료 → 매매 안내·확인 → REGIME 확인 → CSV** 순서로 표시할 모달을 선택합니다. `UiIntentRouter.can_open_modal()`은 중복 열기와 충돌하는 입력을 제한합니다. 모달을 표시한다는 Action의 화면 효과는 이러한 상태 선택과 `AppModalHost` 렌더링으로 완성됩니다.

6. 비동기 명령을 누가 시작하고 결과를 누가 받는가

6.1 작업 요청의 정의와 반환값

기능 파일의 `meta.command`에는 작업 이름, input 계산, 성공 전이, 실패 전이가 선언되어 있습니다. 이곳의 input 함수는 현재 데이터로 요청 인자를 계산할 뿐 Port를 호출하지 않습니다. `UiRegionComposition.command()`와 `connect_commands()`가 이 선언을 루트의 요청 Action과 완료·실패 전이에 연결합니다.

명령 시작 Action은 작업 상태로 향하는 명시적 입력 또는 완료 전이에 붙습니다. **history로 작업 상태에 다시 들어왔다는 이 유만으로 HTTP를 다시 보내지 않습니다.** 타이머는 별도로 state entry에서 시작하되 이미 보유한 요청 슬롯이 있으면 재시작하지 않습니다.

`uiActions.ts`의 `UITransitionResult`는 다음 두 항목을 반환합니다.

```
export interface UITransitionResult {
  readonly snapshot: UiApplicationSnapshot;
  readonly actions: readonly UIActionRequest[];
}
```

이 배열에는 실행 순서가 있습니다. 요청에는 함수·Promise·actor·Port 참조를 넣지 않습니다. STM이 반환하는 값은 “어떤 작업을 어떤 입력으로 실행할 것인가”를 나타내는 데이터입니다.

요청 type	데이터와 실행 의미
<code>run_command</code>	<code>key, token, operation, input</code> ; 지정한 명령 시작
<code>cancel_command</code>	<code>key</code> ; 해당 key의 현재 작업 정리
<code>start_timer</code>	<code>key, token, due_at_ms</code> ; 지정한 만료 시각에 완료 이벤트 전달
<code>cancel_timer</code>	<code>key</code> ; 해당 타이머 취소
<code>stop_all</code>	실행부가 보유한 모든 작업과 타이머 정리

루트의 `assign`이나 즉시 `raise`는 순수 평가 안에서 이미 처리되므로 Controller가 다시 실행하지 않습니다.

`UISTM.read_actions()`는 실행할 데이터 요청만 추출하고, 예상하지 못한 실행 Action은 오류로 처리합니다.

6.2 요청 key와 token

예를 들어 자동매매 시작의 작업 key는 `trading.start_command`입니다. 새 작업은 증가한 token과 함께 등록됩니다.

```
context.requests["trading.start_command"]
= { token: 해당 요청 식별자, status: "pending" }

UITransitionResult.actions에 포함되는 요청 예시
= { type: "run_command", key: "trading.start_command", token: 1,
    operation: "start_trading", input: "type0" }
```

Controller는 `UiCommandExecutor.execute(actions)`를 호출합니다. 실행부는 `operation`에 대응하는 Port 메서드를 호출하고, 작업별 Promise·취소 신호·타이머를 관리합니다. 작업이 끝나면 `deliver` callback으로 Controller의 `handle_event()`에 결과를 전달합니다.

```
command.trading.start_command.done
+ 같은 token
+ source.output = port의 반환값

command.trading.start_command.error
+ 같은 token
+ source.error = 실패 정보
```

실행부는 교체·취소된 작업의 늦은 callback을 차단합니다. 그와 별개로 STM의 결과 guard는 `context.requests[key]?.token === event.token`을 검사합니다. 어떤 결과를 현재 상태에 적용할지는 STM이 판단하며, Controller가 이 가드를 복제하지 않습니다. Port의 동기 예외와 Promise의 비동기 실패는 모두 같은 오류 이벤트 경로를 거칩니다.

6.3 취소의 의미와 화면 이동

상세 내역 읽기는 화면을 나가면 취소되며, `load_trade_history(query, signal)`로 전달된 `AbortSignal`이 HTTP 읽기에 연결됩니다.

설정 저장·REGIME 적용·CSV 생성처럼 이미 시작한 작업은 **화면 이동만으로 중단하거나 재제출하지 않습니다**. Controller가 소유한 실행부가 완료를 받습니다. 명시적 무효화, 같은 key의 새 작업, 해당 작업을 끝내는 다른 전이, 전체 종료에서는 STM의 취소 요청에 따라 실행부의 관리 항목을 정리합니다.

실제 `AbortSignal`은 거래 내역 읽기에 전달합니다. 이미 제출된 쓰기에는 취소 신호를 전달하지 않으며, 실행부의 관리 항목을 지워 늦은 결과 전달을 막는 것으로 backend 쓰기를 되돌렸다고 간주하지 않습니다. 최신 요청 token, 서버 version 검사, 후속 서버 동기화가 각각 자신의 경계에서 결과의 유효성을 확인합니다.

6.4 시간 입력과 타이머의 소유자

`Date.now()`와 `performance.now()`는 Controller가 읽습니다. STM은 전달받은 `now_epoch_ms_monotonic_ms`만 사용하여 상태 규칙과 수신 시각을 계산합니다. 타이머 요청의 `due_at_ms`는 절대 만료 시각입니다. 실행부가 현재 시각과의 차이로 `setTimeout()`을 만들고, STM은 만료 이벤트를 받아 다음 상태를 결정합니다.

CSV 기준일은 일반적인 매 이벤트 시간 입력과 구분합니다. `UISTM.requires_current_date()`가 현재 상태에서 CSV 열기를 순수하게 사전 평가하여 실제 `reset_draft` Action이 선택되는지 확인합니다. 이때 `ui.read_current_date` 표식만 확인하고 snapshot이나 외부 작업을 변경하지 않습니다. Controller는 필요할 때만 날짜 제공 함수를 한 번 호출하고, 그 날짜를 넣어 실제 전이를 평가합니다. 중복 열기로 무시된 입력에서는 날짜를 다시 읽지 않으며 Controller에 CSV 가드를 복제하지 않습니다.

6.5 생성·시작·화면 런타임 종료

생성자에서는 초기 snapshot을 계산하고 초기 Action을 보관합니다. 이 시점에는 API나 타이머를 실행하지 않습니다. `subscribe()`는 현재 snapshot을 즉시 전달하고 해제 함수를 반환합니다. `start()`는 `UISTM.run()`의 초기 Action을 한 번 실행하며, 시작 전에 받은 입력은 이후 큐에서 처리합니다. 중복 `start()`는 작업을 다시 제출하지 않습니다.

시작된 화면 런타임의 `stop()`은 큐·실행 작업·타이머·구독을 정리하고 active snapshot을 `stopped`로 바꿉니다. 이미 final인 snapshot은 `done`을 유지합니다. 폐기된 Controller를 다시 실행하는 대신 새 인스턴스를 생성합니다. 업무상 종료 확인과 네이티브 종료 완료 대기는 11절의 상태 전이로 처리하며, `stop()` 자체가 그 종료 명령을 제출하는 것은 아닙니다.

7. 상단 상태바 — U1·U2·U3

상단의 실제 경로는 `ETIRE_UI_SYSTEM.UPPER_STATUS_BAR`입니다. API 표시, 중지 버튼, 시작 버튼은 그 아래 세 Region입니다.

7.1 API 표시 — U1-01~03

`connectionMachine.ts`는 `api_offline·connecting·api_online·reconnecting`을 사용합니다. 연결·단절 이벤트에 따라 상태와 오류·재연결 정보를 갱신합니다. 재접속 시도와 실제 WebSocket 연결 처리는 adapter가 담당하며 이 Region은 그 결과를 표현합니다.

연결 단절 intent는 연결 표시뿐 아니라 `trading.API_DISCONNECTED`도 전달합니다. 실행 중 단절에 대한 중지 요청과 안내는 매매 제어 정의가 담당합니다.

7.2 시작·중지의 공통 데이터

`tradingRegions.ts`는 시작과 중지를 별도 Region으로 구성합니다. 시작에는 `stopped·running·start_confirmation·starting`과 안내 상태들이 있고, 중지에는 `idle·stop_confirmation·stopping·force_sell_confirmation·force_selling`·완료 대기·복구 청산 상태들이 있습니다.

`STOP_BUTTON.idle`은 “매매가 정지됐다”는 뜻이 아니라 “중지 버튼이 별도 확인이나 처리 작업을 진행하지 않는다”는 뜻입니다. 이때 START Region은 `running`일 수 있습니다. 실제 매매 실행 여부는 공통 `features.trading.is_trading`과 backend lifecycle로 판단합니다.

두 Region이 활성화됐다는 이유만으로 각자 lifecycle을 덮어쓰지 않습니다. 시작·중지 결과와 서버 snapshot이 공통 데이터를 갱신하고, START의 `always` 전이는 이 데이터에 맞춰 `running/stopped`를 정리합니다.

7.3 시작 — U3

상황	상태와 Action
REGIME이 없음	<code>select_regime_notice</code> ; 안내 확인 입력은 REGIME 강조도 요청

상황	상태와 Action
시작 가능한 전략·명령 조건을 충족하지 못함	<code>trading_unavailable_notice</code>
API가 연결되지 않음	<code>api_connection_required</code>
시작 가능한 상태에서 버튼 클릭	선택 REGIME을 기억하고 <code>start_confirmation</code>
확인	REGIME·시작 가능 조건·API를 다시 검사하고 <code>starting</code> 에서 시작 작업 요청
성공	<code>mark_trading_started</code> 와 <code>running</code>
실패	오류를 저장하고 <code>start_confirmation</code>
취소	명령을 제출하지 않고 <code>stopped</code>

정지 상태에서 복구된 열린 포지션이 있으면 Router가 새 자동매매 시작을 막습니다. 자동 재개와 복구 포지션 청산은 별도의 의미로 관리합니다.

7.4 중지·강제 매도·복구 — U2

실행 중 포지션이 없으면 일반 중지 확인, 있으면 강제 매도 및 중지 확인을 표시합니다. 확인은 각각 `stop_trading()` 또는 `force_sell_and_stop()` 작업으로 연결되며, 실제 포지션·미체결 주문에 따른 중지 분기는 backend가 판정합니다.

중지 요청의 수락과 실제 중지 완료는 구분합니다. receipt 또는 서버 통지의 lifecycle이 `terminated/not_started` 인지, 아직 `stopping`이나 `reconciliation_required`인지에 따라 완료와 대기를 나눕니다. 단순히 HTTP가 성공했다는 이유로 포지션이 사라졌다고 표시하지 않습니다.

복구 포지션에는 `liquidate_recovered_position()`과 전용 확인·대기 상태가 있습니다. API 단절 시 중지 Region의 `disconnect_stopping` 작업 후 내부 `trading.DISCONNECT_HANDLED` 이벤트로 시작 Region의 연결 안내를 열며, 안내를 닫은 뒤에도 필요한 종료·조정 대기를 유지합니다.

8. 메인 화면 — REGIME·차트·계좌·탭

8.1 REGIME의 세 Region — R1·R2·R3

`regimeRegions.ts`에서 추천 `recommendation`, 선택 `selection`, 지표 `indicators`가 실제 병렬 Region으로 구성됩니다.

- 추천은 `TYPE_RECOMMENDED`에서 `recommended_regime`을 갱신합니다.
- 지표는 `REGIME_INDICATOR_UPDATED`에서 `metrics`를 갱신합니다.
- 선택은 `candidate_regime`과 `applied_regime`을 구분하여 확인과 적용을 처리합니다.

실행 중인지 정지 중인지와 관계없이 CR-03에 따라 다음 순서를 따릅니다.

```
TYPE_CLICKED
→ remember_candidate
→ type_change_confirmation
→ CONFIRM_TYPE_CHANGE
```

- applying / apply_regime(candidate)
- 성공: apply_candidate → type_selection
- 실패: remember_failure → type_change_confirmation

취소는 **discard_candidate**로 후보만 버립니다. 실패해도 적용값을 후보로 확정하지 않습니다. 서버가 확정한 REGIME 통지는 별도 동기화 경로로 받습니다.

REGIME 선택 안내 후의 강조는 **highlighting**과 4,000ms 타이머로 관리합니다. 강조 여부는 context에 있고 시각 효과는 **RegimePanel.module.css**가 그림입니다. 타이머를 화면 복귀 때마다 처음부터 시작하지 않습니다.

8.2 차트의 여섯 Region — DC1~DC6

표	Region	구현 내용
DC1-01~13	interval	1m/30m/4h/1d 상태와 현재 주기. 기본 주기는 30m이며 초기 옵션을 받을 수 있음
DC2-01~03	indicator_settings	설정창 closed/opened ; opened 내부에 세 지표 Region
DC3-01~02	active_state	backend가 알린 실제 거래 로직 상태 문구 표시
DC4-01~03	viewport	normal/fullscreen 전환; 현재 주기 유지
DC5-01~07	drawing	deactivated/waiting/user_drawing ; 시작·완료·취소·도구 해제
DC6-01~06	line_selection	awaiting_selection/highlighted/context_menu ; hover·메뉴·삭제

indicator_settings.opened 안에는 BB·EMA·거래량의 세 Region이 있습니다. 각 Region의 초기 **choice**는 context의 표시값을 읽어 **on/off**로 정하고, 이후 자신의 선택만 갱신합니다. 이 부분이 IP1·IP2·IP3의 각각 다섯 행입니다.

완성된 선은 **context.features.chart.drawings[interval]**에 저장됩니다. 주기를 바꿔도 다른 주기의 선 배열을 버리지 않으며 돌아오면 해당 주기의 선을 선택해 표시합니다. 이 문서의 “선 저장”은 실행 중 UI context 보존을 뜻하며 디스크 영구 저장을 뜻하지 않습니다.

캔들 조회·실시간 시세와 실제 차트 렌더링은 **useRealtimeChartData.ts**, **PriceChartPanel.tsx**, **LightweightChartSurface.tsx**에 있습니다. chart 상태 정의는 사용자 조작을 관리하고 표시 설정을 이 경로에 전달합니다.

8.3 계좌 정보와 분할 입력 — DI1·DI2·SI·SO

DISPLAY_ACCOUNT_INFO에는 **trading_logic_status·asset_summary·SPLIT_ORDER**가 있습니다. 투자 상태와 자산은 **accountSummaryMachine.ts**의 Action으로 받은 표시값을 갱신합니다. UI가 이 Action에서 거래소 계좌를 직접 조회하거나 손익 원장을 다시 계산하지 않습니다.

SPLIT_ORDER 내부의 매수·매도 입력은 각각 **scale_in.SCALE_IN_ORDER**와 **scale_out.SCALE_OUT_ORDER**입니다. 두 입력은 모두 실제 활성 상태 경로를 갖고 각각 **ready/saving/failed**

를 표현합니다.

저장 요청은 공통 `pending_side·pending_percentage`와 `split_order.update_split_order_command` 슬롯을 사용합니다. `prepare_scale_in` 또는 `prepare_scale_out` Action은 해당 비율을 즉시 화면 데이터에 반영합니다. 이어서 같은 입력에 연결된 명령 시작 Action이 요청 데이터를 반환하고, Controller 소유 실행부가 `update_split_order(side, percentage)`를 호출합니다. 연속 입력은 새 요청으로 교체하고 오래된 완료는 적용하지 않습니다. 다른 방향의 표시값은 보존합니다.

따라서 두 입력 **Region**의 상태 분리와 저장 요청의 독립 병렬 실행 보장은 서로 다른 사항입니다. 이 구조는 최신 입력을 추적하는 공통 요청 정책을 사용합니다. 실패하면 입력값과 오류를 표시하며, 서버 snapshot이 도착하면 서버 기준 비율로 동기화하고 기존 요청을 무효화합니다.

8.4 최근 체결·실시간 지표 — M4-01~09

TRADER_PANEL은 `trade_history_displayed`와 `realtime_indicator_displayed` 중 하나입니다. 체결 이벤트는 어느 탭에서도 `prepend_trade`로 최근 목록 앞에 추가됩니다. 전략별 지표는 별도 context에 유지되므로 탭 변경이 체결 수신을 막지 않습니다.

표의 체결 파일 저장은 backend의 `TradeHistoryController._publish_completed_trade()`와 `trade_history_repository.py`가 담당합니다. UI는 저장된 내역을 초기 snapshot과 통지로 받아 표시합니다. 탭을 바꿀 때 마다 UI가 거래 파일을 직접 읽거나 쓰지는 않습니다.

전략 지표의 경과 시간 표시에는 Controller가 전달한 단조 시각을 사용합니다. 같은 지표가 재전송되면 기존 `strategy_indicators_received_at`을 유지하고, 새 지표일 때만 수신 시각을 바꿉니다. 이 값은 화면용이며 backend의 전략 시간이나 거래 판단을 변경하지 않습니다.

9. 상세 화면 — 요약·결합 필터·CSV

9.1 상세 요약의 네 Region — D1~D4

ACCOUNT_DETAILS의 `profit_rate·sell_performance·eth_holdings·daily_trading_fee`는 동시에 활성화됩니다. 정의는 `tradeHistorySummaryMachine.ts`에 있습니다.

서버가 확정된 수익률·매도 성과·보유량·수수료를 각각 갱신합니다. 매도 성과와 보유량을 함께 담은 `SELL_ORDER_EXECUTED`는 관련 두 Region에 작용할 수 있습니다. live 경로는 서버 메시지를 적절한 요약 intent로 변환하므로, 모든 체결 통지를 모든 요약 Region에 같은 문자열로 전달한다고 이해하면 안 됩니다.

요약은 거래 표의 현재 필터와 구분됩니다. 필터 변경 조회는 `should_publish_summary: false`로 요약을 덮어쓰지 않습니다. 요약 갱신이 허용된 조회라도 요청 당시 `summary_revision`과 현재 revision이 같을 때만 반영합니다.

9.2 기간과 거래 종류, 그리고 조회 작업 — TD2·TD3

`historyFilterRegions.ts`가 기간과 거래 종류를 실제 상태로 구성합니다.

```
TRADING_DETAILS.PERIOD.selection = today / last7days / last30days / all
TRADING_DETAILS.SIDE              = all / buy / sell
TRADING_DETAILS.PERIOD.query      = idle / loading / ready / empty / failed
```

선택 상태는 따로 유지하지만 조회는 `context.features.trade_history`의 `period`와 `side`를 결합하여 한 요청으로 보냅니다. 기간 이벤트는 기간만 바꾸고, 거래 종류 이벤트는 거래 종류만 바꿉니다. 같은 이벤트에 선택 Region과 query 상태가 함께 반응하므로 선택 표시와 요청 조건을 일치시킵니다.

실제 명령은 `load_trade_history({ period, side }, signal)`이고, backend `trade_history.py`의 조회 경로에 연결됩니다.

- 최초 진입은 오늘·전체로 초기화합니다.
- 재진입은 선택한 기간·거래 종류를 유지하면서 다시 조회합니다.
- `ready/empty/failed`에서 필터를 바꿀 수 있습니다. `loading` 중 변경 입력은 적용하지 않습니다.
- 화면 이탈 시 현재 조회를 취소하고 늦은 응답을 무시합니다.
- 조회 중 체결이 들어오면 `refresh_pending`을 세워 첫 결과를 적용하지 않고 같은 조건을 다시 조회합니다.
- 표시 중인 `ready/empty`에서는 전달받은 현재 시각으로 다음 KST 자정의 만료 시각을 계산하고 타이머를 요청합니다. Controller 소유 실행부가 타이머를 만들며, 상세 화면 이탈 시 이 조회용 타이머도 취소합니다.

9.3 CSV 외부 상태 — TD4-01~09

`CSV_EXPORT`는 `closed` → `editing` → `exporting` → `complete/error` 흐름입니다. 실제 설정 계층은 `csvExportRegions.ts`, 입력·검증·명령 정의는 `csvExportMachine.ts`에서 읽습니다.

새 팝업 열기로 수락된 `CSV_EXPORT_CLICKED`는 `reset_draft`를 수행합니다. Controller가 그 경계에서 읽어 전달한 KST 날짜로 오늘 범위와 기본 파일명을 만들고, 경로는 미선택으로 시작합니다. 날짜 제공 함수를 STM 안에서 호출하지 않으며, 중복 열기에서는 기존 draft를 다시 초기화하지 않습니다. 상세 목록의 필터를 CSV 기본값으로 복사하지 않습니다.

`EXPORT_CSV`에서 경로·파일명·기간 검증에 실패하면 `validate_draft`로 오류만 표시하고 `editing`에 남습니다. 통과하면 세 입력 Region을 함께 빠져나와 `exporting`으로 이동하고 한 번의 파일 생성 작업을 시작합니다. 완료는 `complete`, 실패는 `error`이며, 실패 확인은 draft를 유지한 `editing`으로 돌아갑니다. 완료 확인은 `closed`입니다.

9.4 CSV 입력 내부의 세 Region — CR1·CR2·CR3

Region	상태와 처리
경로 <code>file_browser</code>	<code>closed/opened</code> . <code>pick_csv_directory()</code> 결과가 유효한 문자열이면 저장, 취소 <code>null</code> 이면 기존 경로 유지. 오류는 오류 정보로 표시
기간 <code>period</code>	<code>today/weekly/monthly/custom</code> . <code>custom</code> 내부는 <code>idle/start_calendar/end_calendar</code>
파일명 <code>file_name</code>	<code>DEFAULT_FILE_NAME/editing/FILE_NAME_WRITTEN</code> . 진입 시 <code>restore</code> 분기가 확정 여부를 읽음

유효한 날짜를 선택하면 값을 반영하고 달력은 열린 상태로 유지합니다. 시작일이 종료일보다 늦거나 종료일이 시작일보다 이르면 입력을 반영하지 않고 오류를 표시합니다. 달력 외부 클릭으로 닫습니다.

기간이나 날짜만 바꾼다고 파일명을 자동으로 바꾸지 않습니다. 파일명은 입력 draft와 확정값을 구분하고, 유효한 Enter 또는 포커스 해제로 확정하면서 `.csv` 이름을 정규화합니다. 잘못된 파일명은 `editing`에 남습니다.

`file_name_committed`는 실패 후 설정 화면으로 돌아왔을 때 기본값 상태인지 확정 상태인지 구분합니다.

폴더 선택은 Rust의 `choose_csv_export_directory()`에 연결됩니다. 실제 CSV 파일 생성 경로는 다음과 같습니다.

STM의 CSV 명령 요청

- UIApplication / UiCommandExecutor
- BackendUiAdapter.export_csv()
- backend CSV export route
- TradeHistoryController.export_csv()
- CSVFileGateway.write_csv()
- 파일 생성 결과
- Controller 큐 → STM의 CSV_EXPORT.complete 또는 error

각 구현은 **adapter** → **route** → **controller** → **파일 writer**에서 확인합니다. UI는 검증과 요청·결과 표시를 담당하고, CSV 바이트 기록은 backend가 담당합니다.

10. 화면 이동과 실제 deep history — ES2

교수님, **SHOW_TRADE_HISTORY**는 화면 이동 이벤트와 상세 조회 진입 이벤트를 한 묶음으로 보냅니다. 화면은 **TRADING_DETAILS**로 전이하고, 해당 화면의 Region들이 활성화됩니다.

메인 내부에는 다음과 같은 실제 deep history 노드가 있습니다.

```
HISTORY: {
  id: 'MAIN_SCREEN_HISTORY',
  type: 'history',
  history: 'deep',
  meta: {
    spec_ids: ['ES2-03', 'H*'],
  },
},
```

복귀 target은 **#MAIN_SCREEN_HISTORY**입니다. XState는 메인 하위의 차트 주기·지표 설정·그리기 상태·탭 등 활성 상태 구성을 복원합니다. 상세 화면에도 **DETAILS_HISTORY**가 있어 그 내부 상태를 기억하지만, 상세 목록 조회는 별도의 **ENTER_TRADE_HISTORY** 입력에 따라 새로 수행합니다.

10.1 history는 서버 데이터를 과거로 되돌리지 않습니다

history가 기억하는 것은 하위 상태 구성입니다. 계좌 값이나 체결 내역의 과거 context 전체를 저장했다가 덮어쓰는 기능이 아닙니다.

비활성 화면의 서버 통지는 **UiRegionComposition**이 루트에 구성한 fallback handler가 받습니다. 해당 기능이 비활성이라는 guard를 검사한 다음 현재 **features** 데이터를 갱신합니다. 그래서 메인으로 돌아오면 사용자 설정은 유지되면서 최신 계좌·체결·전략 값을 표시합니다.

retained와 **retained_details**는 화면 밖 기능의 view를 읽기 위해 저장한 상태값입니다. **UISTM.handle()**이 전이 직전 **snapshot.value**를 **evaluation.previous_value**로 전달하고, 화면의 exit Action이 이를 보관합니다. 순수 평가 안에서 **self.getSnapshot()**에 의존하면 실제 이전 상태 대신 초기 상태를 읽을 수 있으므로 사용하지 않습니다. 이 보관값은 실제 deep history 전이를 대신 실행하는 별도 상태 머신이 아닙니다.

10.2 화면 밖에서 작업이 끝난 경우

REGIME 적용을 요청한 뒤 화면이 비활성화되고 그동안 작업이 끝나는 경우를 생각하면 다음 순서입니다.

1. Controller 소유 실행부가 결과를 Controller 큐에 전달하고 STM이 다음 이벤트로 평가합니다.
2. fallback handler가 token과 결과 guard를 확인합니다.
3. 결과 Action은 즉시 데이터에 반영하고, 복귀 때 필요한 target 전이를 **deferred**에 보관합니다.
4. 메인 재진입 시 해당 resume 이벤트를 올립니다.
5. resume handler는 상태 target만 적용합니다. 이미 반영한 결과 Action이나 HTTP 작업을 다시 실행하지 않습니다.

상태에 다시 진입하면서 표시용 entry Action은 실행될 수 있습니다. 명령 시작은 별도의 명시적 전이에 연결되어 있으므로, 이 entry 처리와 HTTP 재제출을 구분합니다.

서버의 확정된 상태가 진행 요청을 무효화해야 하는 경우에는 해당 요청과 대기 중 target도 정리하여 오래된 완료가 최신 데이터를 덮지 못하게 합니다.

강조 타이머는 화면이 숨겨져도 원래 만료를 계속 추적합니다. 돌아올 때 만료됐다면 강조를 끝낸 상태로 정리하며, 단순 복귀 때문에 4초를 새로 주지 않습니다. React 컴포넌트만 소유한 일시적 값까지 모두 history가 저장한다는 의미는 아닙니다.

11. 프로그램 종료 — ES3

실제 OS 창 닫기는 `use_desktop_window_lifecycle()`이 `preventDefault()`로 막고 `APP_EXIT_CLICKED` intent를 전달합니다. Router는 `features.trading`에서 포지션 정보를 읽어 `app_exit.EXIT_CLICKED`를 보냅니다.

포지션이 있으면 `force_sell_exit_confirmation`, 없으면 `exit_confirmation`입니다. 확인하면 `shutting_down`으로 이동하여 다음 작업을 요청합니다.

```
shutdown_application(context.had_open_position)
```

이 boolean은 포지션 청산을 확인했다는 정보를 전달합니다. UI 종료 Region이 매도 요청과 종료 요청을 따로 연속 제출하는 구조로 읽으면 안 됩니다. `BackendUiAdapter.shutdown_application()`이 종료 준비를 조정합니다.

```
종료 확인
→ EXIT.shutting_down
→ shutdown_application(liquidation_confirmed)
→ backend 종료 상태 확인 및 /v1/shutdown/prepare 작업
→ 준비 결과가 ready인지 확인
→ /v1/shutdown 수락
→ 네이티브 backend 프로세스의 실제 정상 종료 확인
→ 루트 UI_FINAL_STATE
→ AppViewModel.app_exit.is_final
→ use_desktop_window_lifecycle()의 destroy()
```

HTTP 202는 요청 수락이므로 그 응답만으로 창을 닫지 않습니다. adapter는 프로세스 종료 확인을 기다립니다. 오류는 `appExitMachine.ts`의 다음 분기로 연결됩니다.

결과	UI 상태
청산 동의 필요	<code>force_sell_exit_confirmation</code>
프로세스 종료 확인 시간 초과	<code>shutdown_exit_recovery</code>
종료 결과 불명확	<code>shutdown_outcome_recovery</code>
프로세스 비정상 종료	<code>sidecar_exit_failure</code>
그 외 종료 준비 실패	오류를 보존한 <code>exit_confirmation</code>
정상 종료 완료 또는 허용된 상태의 정상 종료 통지	루트 <code>UI_FINAL_STATE</code>

종료가 이미 수락됐거나 결과가 불명확한 복구 상태에서 “취소”는 정상 거래 화면으로 돌아가게 하지 않습니다. 비정상 종료 안내는 확인 후 루트 final로 끝나며 이미 종료된 backend에 새 종료 명령을 보내지 않습니다.

기능 정의 안의 `ui_final_state` target은 조립 단계에서 `**#UI_FINAL_STATE**`로 연결됩니다. 따라서 최종 도달 점은 EXIT만의 종료가 아니라 전체 UI 루트의 final입니다. STM은 `requests.deferred`를 비우고 `stop_all`을 반환합니다. Controller가 남은 실행 작업을 정리한 뒤 final snapshot을 발행하며, 이후 입력과 낮은 결과는 처리하지 않습니다. 브라우저 실행은 네이티브 창 port가 없으므로 desktop의 `destroy()` 경로가 실행되지 않습니다.

12. 실제 코드로 따라가는 자동매매 시작

교수님, 적용 REGIME이 있고 API가 연결됐으며 시작이 허용된 상태라고 가정하겠습니다. 아래 코드는 현재 파일에서 발췌했습니다. 요청 데이터 예시는 실행 코드를 대신하는 것이 아니라 STM과 Controller 사이에서 전달하는 값을 설명합니다.

12.1 React callback → Store → Controller

`App.tsx`의 시작 버튼 연결입니다.

```
onStartRequested={() => controller.dispatch({ type: 'START_TRADING_CLICKED' })}
```

여기서 `controller`는 UI Store를 통해 전달되는 제어 객체이며 Python `TradingController`가 아닙니다. 클릭하면 Store의 `dispatch()`를 거쳐 `UiStateController.dispatch()`로 들어갑니다. Store에 남아 있는 `UiApplicationFacade` import도 같은 클래스를 가리킵니다.

Controller는 Router가 수락한 이벤트를 큐 진입점에 전달합니다.

```
dispatch(intent: UiApplicationIntent): boolean {
  const router = new UiIntentRouter(this.get_snapshot());
  const accepted = router.dispatch(intent);
  if (accepted && router.events.length) this.handle_event({ type:
'ui.batch', events: router.events,
...(intent.type === 'BACKEND_SNAPSHOT_SYNCHRONIZED' ? {
server_snapshot: intent.snapshot } : {}) });
  return accepted;
}
```

이때 만들어지는 내부 이벤트는 `trading.START_BUTTON_CLICKED`이고 `source`에 `regime`과 `is_online`이 들어갑니다. `UiIntentRouter`는 `this.read('regime')`와 `this.read('connection')`으로 현재 `snapshot`의 값을 읽습니다.

가드가 통과하면 STM이 `START_BUTTON.start_confirmation`으로 전이합니다. 아직 `backend` 시작 명령을 제출한 것은 아닙니다.

12.2 확인 입력 → starting과 명령 요청

`AppModalHost.tsx`의 확인 callback은 다음 intent를 보냅니다.

```
onConfirm={() => controller.dispatch({ type: 'START_TRADING_CONFIRMED' })}
```

`Router`는 API 연결 상태를 다시 읽어 `trading.START_CONFIRMED`를 만듭니다. 시작 확인 상태의 가드가 `REGIME`·시작 가능 여부·API를 다시 검사한 뒤 `starting`으로 전이합니다.

`tradingCommandMachine.ts`의 `starting.meta`에는 다음 선언이 있습니다.

```
command: {
  id: 'start_command',
  src: 'start_trading',
  input: ({ context }: { context: TradingCommandContext }) =>
    context.selected_regime as RegimeType,
  onDone: {
    target: 'running',
    actions: 'mark_trading_started',
  },
  onError: {
    target: 'start_confirmation',
    actions: 'remember_failure',
  },
},
```

`input` 함수에 전달되는 `context`는 조립부가 추출한 `features.trading`입니다. `selected_regime` 값을 계산하여 요청 데이터에 담고, 함수 자체는 `Controller`로 넘기지 않습니다. `onDone/onError`는 STM에 남아 있는 결과 전이 규칙입니다.

조립부는 `starting`으로 향하는 명시적 전이에 작업 시작 Action을 붙입니다. 평가 결과에는 다음 형태의 데이터가 포함됩니다. 아래 `token`은 설명을 위한 예시이며 실제로는 `request_sequence`에서 부여합니다.

```
{ type: "run_command", key: "trading.start_command", token: 1,
  operation: "start_trading", input: "type0" }
```

12.3 순수 전이 결과를 Controller에 반환

`UISTM.handle()`의 구현입니다.

```

handle(event: UiDomainEvent, input: UIEvaluationInput): UITransitionResult
{
  if (this.snapshot.status !== 'active') return this.result([]);
  const [snapshot, actions] = transition(this.machine, this.snapshot, {
    type: 'ui.evaluate',
    evaluation: { ...input, previous_value: this.snapshot.value },
    events: [event],
  });
  this.snapshot = snapshot;
  return this.result(this.read_actions(actions));
}

```

XState가 같은 처리 단위의 내부 전이를 모두 계산한 뒤 STM은 새 snapshot을 보관하고 요청 목록을 반환합니다. 평가만으로 Port를 호출하지 않습니다. `previous_value`는 이번 전이 직전 화면 상태를 명시적으로 전달하는 값입니다.

Controller의 `drain()`은 큐에서 이벤트 하나를 꺼내 `read_time(event)`와 함께 이 메서드에 전달하고, 반환값을 `apply()`로 처리합니다. 따라서 외부 명령을 시작할 때 `get_snapshot()`으로 읽는 값은 이미 완성된 `starting` snapshot입니다.

12.4 Controller 소유 실행부 → command port

`UiCommandExecutor`는 명령 요청마다 `PendingWork`를 만들고 key별 `pending` Map에 보관합니다. 이 객체에는 `tokenAbortController`와 필요할 때 타이머 핸들이 들어갑니다. 같은 key의 기존 작업은 먼저 정리합니다.

명령 실행과 결과 연결 부분은 다음과 같습니다.

```

void this.run(action, work.abort.signal).then(
  output => this.finish(action.key, work, 'done', output),
  error => this.finish(action.key, work, 'error', error),
);

```

`run()`의 `switch (request.operation)`에서 자동매매 시작은 다음 분기에 연결됩니다.

```

case 'start_trading': return this.port.start_trading(request.input);

```

live 구성의 `port`는 `BackendUiAdapter`입니다. 실제 Port 참조를 가진 곳은 이 실행부이며, 기능별 machine에는 `command_port` 클로저나 `fromPromise()`가 없습니다. `run()`이 async 함수이므로 동기 예외도 거부된 Promise가 되어 같은 오류 전달 경로로 들어갑니다.

완료·실패를 전달하는 구현은 다음과 같습니다.

```

private finish(key: string, work: PendingWork, kind: 'done' | 'error',
value: unknown): void {
  if (this.pending.get(key) !== work) return;
  this.pending.delete(key);
}

```

```

    this.deliver({ type: `command.${key}.${kind}`, token: work.token,
                  source: kind === 'done' ? { type: 'command.done', output: value } :
                  { type: 'command.error', error: value } });
  }

```

`pending`의 객체 동일성 검사는 교체·취소된 작업의 callback을 차단합니다. `deliver`는 Controller 생성 시 연결한 `event => this.handle_event(event)`입니다. 결과를 React나 Python에 직접 돌려주는 것이 아니라 다음 STM 평가를 위한 이벤트로 큐에 넣습니다. 어떤 결과를 현재 상태에 수락할지는 별도로 STM의 token 가드가 검사합니다.

12.5 HTTP와 backend

`BackendUiAdapter.start_trading()`은 적용 REGIME 일치를 확인하고 `expected_version`을 사용하여 `POST /v1/trading/start`를 보냅니다. 응답의 `running` 상태·session ID·version을 검증하여 receipt를 반환하거나 오류를 던집니다.

backend의 `trading.py`가 `TradingController.start_trading()`으로 연결됩니다. Python의 거래 STM과 실제 주문 로직은 이 backend 경계 안에서 동작합니다. UI machine은 Python 객체를 직접 갖고 있지 않습니다.

12.6 결과 → 다음 STM 전이 → 화면

Controller가 완료 이벤트를 큐에서 꺼내 STM을 평가하면, STM은 `command.trading.start_command.done`의 token을 확인하고 `mark_trading_started`를 수행하여 `running`으로 전이합니다. 실패라면 `remember_failure`와 `start_confirmation`이 적용됩니다.

최종 snapshot이 STM에 반영된 뒤 Controller는 다음 순서로 작업과 발행을 처리합니다.

```

private apply(result: UITransitionResult): void {
  this.executor.execute(result.actions);
  if (!this.is_started) return;
  this.listeners.forEach(listener => {
    try { listener(result.snapshot); }
    catch (error) { queueMicrotask(() => { throw error; }); }
  });
}

```

각 객체의 역할을 이어 쓰면 다음과 같습니다.

Port의 Promise 완료

- `UiCommandExecutor.finish()`
- `Controller.handle_event()`의 직렬 큐
- `UISTM.handle()`: guard·내부 Action·상태 전이 계산
- `UITransitionResult` 반환
- `Controller.apply()`: 요청 실행 후 최종 snapshot 발행
- Store의 최신 `AppViewModel` 캐시 갱신
- React 구독 알림
- 시작 버튼·`pending` 표시·확인창 렌더링

실행이나 구독 callback 중 추가로 들어온 이벤트는 현재 발행이 끝난 뒤 처리합니다. 이 흐름에서 UI Action이 DOM의 버튼 문구를 직접 바꾸는 것은 아닙니다. `is_trading.is_pending.active_modal` 같은 화면 모델을 계산하고 React가 그 값을 그립니다. 별도로 도착하는 backend lifecycle 통지도 같은 Controller 큐를 거쳐 STM의 공통 매매 데이터에 반영됩니다.

13. 실제 코드로 보는 Region 독립성과 여러 Action

13.1 EMA만 끄는 Action

`chartMachine.ts`의 Action입니다.

```
hide_ema9: assign({
  indicators: ({ context }) => ({
    ...context.indicators,
    ema9: false,
  }),
}),
```

`...context.indicators`가 기존 BB·거래량 값을 유지하고 `ema9`만 바꿉니다. 조립부는 이 `assign`을 `context.features.chart`의 갱신으로 연결합니다. 지표의 상태 전이는 EMA Region에서 일어나고, 다른 Region의 활성 상태는 유지됩니다.

루트 행동 테스트는 실제 Controller-STM 경계에서 지표 설정창을 열고 각 지표를 켜고 끄면서 다른 지표 값이 유지되는지 검사합니다.

13.2 한 이벤트의 여러 Action

`tradeHistoryMachine.ts`의 조회 완료 분기입니다.

```
{
  guard: 'has_records',
  target: 'ready',
  actions: ['store_records', 'ui_publish_summary'],
},
```

XState가 `store_records`와 `ui_publish_summary`를 배열 순서대로 처리합니다. 첫 Action은 목록을 갱신합니다. 루트 조립부는 `ui_publish_summary`를 다음 조건을 확인하는 루트 `assign`으로 연결합니다.

```
output.publish_summary가 true인가?
요청 당시 request_summary_revision이 현재 summary_revision과 같은가?
→ 둘 다 맞을 때 features.trade_history_summary.summary 반영
```

두 Action은 모두 STM 내부의 순수 데이터 변경입니다. 한 루트 context 안에서 목록과 허용된 요약 변경을 함께 처리하며, 외부 callback이나 별도 요약 실행 인스턴스를 호출하지 않습니다.

13.3 CSV의 가드 실패와 성공

csvExportMachine.ts의 전이는 다음과 같습니다.

```
EXPORT_CSV: [
  { guard: 'is_csv_draft_valid', target: 'exporting' },
  { actions: 'validate_draft' },
],
```

검증 성공은 **exporting** 상태로의 전이와 **export_csv** 요청 데이터 반환으로 이어지고 Controller가 실제 작업을 제출합니다. 검증 실패는 두 번째 분기의 **validate_draft**만 실행하여 오류를 저장하고 입력 Region을 유지합니다. “Action이 실행됐다”는 사실만으로 파일을 썼다고 판단할 수 없는 이유입니다. 어떤 Action이 context를 바꾸는지, 어떤 전이가 외부 명령 요청을 반환하는지 구분해야 합니다.

14. 구현과 설명을 확인하는 방법

교수님, 이 문서의 상태 계층과 책임 분리는 선언의 이름뿐 아니라 실제 전이 결과와 실행 효과를 검사하는 테스트에 연결됩니다.

확인 대상	근거
최상위 3 Region, 메인·상세 및 하위 병렬 계층	uiApplicationMachine.test.ts
182개 표 ID별 상태·표시·명령 효과	uiEventActionCoverage.test.ts
입력 수락·모달·서버 데이터와 기존 Facade 계약	UiApplicationFacade.test.ts
화면 밖 결과 처리, history 복귀, 재제출 방지, 늦은 결과 무시	uiApplicationMachine.test.ts
외부 호출·실시간 시계 없는 결정적 전이, 데이터 요청·token·history	UISTM.test.ts
snapshot 반영과 요청/발행 순서, 재진입, 중복 시작, CSV 날짜 경계	UIStateController.test.ts
실제 AbortSignal, 작업 교체, 동기 예외·비동기 실패, 타이머·종료 정리	UiCommandExecutor.test.ts
STM 런타임 의존성, Port·실행 actor·타이머·현재 시각 조회의 재유입 방지	uiArchitecture.test.ts
변경 전후의 상태·발행·ViewModel·명령 인자/횟수/순서 비교	uiSeparationReplay.test.ts
실제 로컬 backend 계약·초기화 연결	createLiveUiApplication.process.test.mjs 와 live bootstrap 테스트

[uiEventActionCoverage.test.ts](#)의 [register_behavior_scenario\(\)](#)는 설계 ID와 검증 callback을 함께 등록합니다. [create_spec_ids\(\)](#)는 접두사와 행 개수로 연속된 설계 ID를 만들고, [create_root_test_application\(\)](#)은 실제 [UIStateController](#)와 STM을 준비합니다. helper에 남아 있는 [actor.send/getSnapshot](#) 이름은 Controller를 감싼 테스트 도구이며 실행 중인 XState actor가 아닙니다.

각 시나리오는 입력·상태·표시값·명령 횟수와 결과를 검사합니다. 마지막에는 등록된 시나리오들이 설계 표의 182개 ID를 모두 포함하는지 확인합니다. 여러 설계 행을 한 시나리오가 검증할 수 있으므로 182개 ID와 테스트 개수는 동일하지 않습니다. 일부 시나리오는 내부 이벤트를 직접 전달하므로, 실제 사용자 intent의 화면 제한·변환은 Controller의 기존 Facade 테스트와 루트 통합 테스트를 함께 읽어 확인해야 합니다.

기능별 기존 테스트는 [createFeatureTestController.ts](#)를 통해 실제 STM-Controller 경계에서 실행합니다. 기대한 상태·표시·결과는 유지하고, actor 실행에 의존하던 테스트 준비만 바꿨습니다. 자정 테스트도 STM에 지연 callback을 주는 대신 고정 시각을 사용합니다.

14.1 변경 전후 비교와 최근 검증 결과

기존 commit은 [58727e2901a04a2a0faeb6dd594b94dc51b232d8](#)입니다. 책임 분리 전에 고정 시간·가짜 Adapter·제어 가능한 Promise로 세 시나리오의 기존 기록을 만들었으며, 변경 뒤 기대 기록을 갱신하지 않고 재생하여 일치하는지 확인했습니다.

비교 시나리오	보존한 관찰값
화면 이동·REGIME·시작·분할 입력·필터·CSV·날짜	모든 snapshot 발행, 상태 경로, ViewModel, 입력 수락 여부, 명령 인자·횟수·순서
연속 쓰기 교체·숨겨진 완료·최신 실패·오래된 성공·복귀	완료 순서가 달라져도 최신 결과와 복귀 동작 유지
숨겨진 강조·KST 자정 조회·강조 만료 후 복귀	자정 재조회와 원래 만료 시각 유지

2026-09-17 책임 분리 구현 검증에서 **전체 UI 60개 파일, 660개 테스트가 통과했습니다**. 여기에는 기존 로컬 Python backend 프로세스 연동과 182개 ID 행동 검증이 포함됩니다. 세 기존 기록 모두 일치했고, TypeScript 타입 검사와 Vite production build도 통과했습니다. build에는 503.31 kB chunk가 기본 경고 기준 500 kB를 넘는다는 경고가 남아 있습니다.

세부 명령·기존 기록 SHA-256·결과는 [검증 기록](#)과 [책임 분리 구현 보고서](#)에 있습니다. 이 수치는 문서 개정 전에 완료한 구현 검증 결과이며, 문서 수정만을 이유로 전체 실행 테스트를 다시 수행한 것은 아닙니다. 이번 문서 개정에서는 코드 발체 15개, 파일·줄 링크 441개와 182개 ID 색인을 현재 소스에 대조했습니다. 32개 표의 ID·event·설계 원문 연결은 유지하고 행동 테스트의 등록 위치를 현재 줄 번호로 갱신했습니다.

14.2 검증 범위와 별도 발견 사항

실제 Binance 주문, 전체 backend 테스트군, 네이티브 바이너리 패키징·실기 종료는 이번 검증에 포함하지 않았습니다. 네이티브 종료 완료를 기다리는 UI 계약은 기존 테스트로 확인했습니다. 회귀 시나리오의 일치가 모든 가능한 이벤트 순서에 대한 수학적 증명을 뜻하지는 않습니다.

추가 Communication 대응표 검사는 UI 관련 참조 132개 모두 유효했지만, 기존 backend 테스트 이름을 가리키는 낡은 참조 3개 때문에 전체 검사는 통과하지 않았습니다. 해당 항목은 [별도 발견 사항](#)에 기록했고 책임 분리와 함께 수정하지 않았습니다. backend 거래 정책·저장 형식·네이티브 종료 정책·화면 스타일과 원래 Event-Action ID는 이번 구조 변경에서 유지했습니다.

15. 전체 182개 ID의 구현·행동 검증 색인

아래 색인은 현재 설계 표 순서입니다. 각 묶음에 **실제 루트 상태 경로와 실행 정의 파일**을 적고, 각 행에는 설계 원문과 그 동작을 실행하는 테스트 위치를 연결합니다.

None은 초기 진입 또는 이벤트 없는 가드 분기입니다. 초기 표시에는 **initial**·context 초기값·live snapshot·최초 렌더링이 함께 관여합니다. 표의 명령 성공·실패 event는 구현에서 명령 결과 이벤트와 lifecycle 판정으로 대응될 수 있습니다.

테스트 링크가 같은 행들은 하나의 시나리오에서 초기값, 입력, 취소, 성공·실패 등을 함께 검사합니다. 반복문 안의 링크는 해당 ID의 출발 상태·도착 상태 조합을 생성하는 위치입니다.

U1 — API 상태

실제 경로: **ETIRE_UI_SYSTEM.UPPER_STATUS_BAR.API_DISPLAY**

실행 정의: [connectionMachine.ts](#)

설계 ID	표의 event	실행 검증
U1-01	None	행동 시나리오
U1-02	API_CONNECTED	행동 시나리오
U1-03	API_DISCONNECTED	행동 시나리오

U2 — 매매 중지

실제 경로: **ETIRE_UI_SYSTEM.UPPER_STATUS_BAR.STOP_BUTTON**

실행 정의: [tradingRegions.ts](#) · [tradingCommandMachine.ts](#)

설계 ID	표의 event	실행 검증
U2-01	None	행동 시나리오
U2-02	STOP_BUTTON_CLICKED	행동 시나리오
U2-03	STOP_BUTTON_CLICKED	행동 시나리오
U2-04	STOP_BUTTON_CLICKED	행동 시나리오
U2-05	STOP_CONFIRMED	행동 시나리오
U2-06	STOP_CANCELED	행동 시나리오
U2-07	FORCE_SELL_AND_STOP_CONFIRMED	행동 시나리오
U2-08	FORCE_SELL_AND_STOP_SUCCEEDED	행동 시나리오
U2-09	FORCE_SELL_AND_STOP_FAILED	행동 시나리오
U2-10	FORCE_SELL_AND_STOP_CANCELED	행동 시나리오

U3 — 자동매매 시작

실제 경로: **ETIRE_UI_SYSTEM.UPPER_STATUS_BAR.START_BUTTON**

실행 정의: [tradingRegions.ts](#) · [tradingCommandMachine.ts](#)

설계 ID	표의 event	실행 검증
U3-01	None	행동 시나리오
U3-02	START_BUTTON_CLICKED	행동 시나리오
U3-03	START_BUTTON_CLICKED	행동 시나리오
U3-04	START_BUTTON_CLICKED	행동 시나리오
U3-05	START_CONFIRMED	행동 시나리오
U3-06	START_CONFIRMED	행동 시나리오
U3-07	START_CANCELED	행동 시나리오
U3-08	SELECT_REGIME_NOTICE_CONFIRMED	행동 시나리오
U3-09	API_CONNECTION_NOTICE_CONFIRMED	행동 시나리오
U3-10	FORCE_SELL_AND_STOP_SUCCEEDED	행동 시나리오
U3-11	STOP_CONFIRMED	행동 시나리오
U3-12	API_DISCONNECTED	행동 시나리오

ES2 — 화면 이동

실제 경로: `ETIRE_UI_SYSTEM.SCREEN`

실행 정의: `uiApplicationMachine.ts` · `uiApplicationIntents.ts`

설계 ID	표의 event	실행 검증
ES2-01	None	행동 시나리오
ES2-02	SHOW_ALL_TRADING_DETAILS	행동 시나리오
ES2-03	BACK_TO_MAIN_SCREEN	행동 시나리오

R1 — REGIME 추천

실제 경로: `ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.REGIME_PANEL.recommendation`

실행 정의: `regimeRegions.ts` · `regimeMachine.ts`

설계 ID	표의 event	실행 검증
R1-01	None	행동 시나리오
R1-02	TYPE_RECOMMENDED	행동 시나리오

R2 — REGIME 선택

실제 경로: `ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.REGIME_PANEL.selection`

실행 정의: `regimeRegions.ts` · `regimeMachine.ts`

설계 ID	표의 event	실행 검증
R2-01	None	행동 시나리오
R2-02	TYPE_CLICKED	행동 시나리오
R2-03	TYPE_CLICKED	행동 시나리오
R2-04	CONFIRM_TYPE_CHANGE	행동 시나리오
R2-05	CANCEL_TYPE_CHANGE	행동 시나리오

R3 — REGIME 지표

실제 경로: `ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.REGIME_PANEL.indicators`

실행 정의: `regimeRegions.ts` · `regimeMachine.ts`

설계 ID	표의 event	실행 검증
R3-01	None	행동 시나리오
R3-02	REGIME_INDICATOR_UPDATED	행동 시나리오

DC1 — 차트 주기

실제 경로: `ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.DISPLAY_CHART.interval`

실행 정의: `chartMachine.ts`

설계 ID	표의 event	실행 검증
DC1-01	None	행동 시나리오
DC1-02	1_M_BUTTON_CLICKED	행동 시나리오
DC1-03	4_H_BUTTON_CLICKED	행동 시나리오
DC1-04	1_DAY_BUTTON_CLICKED	행동 시나리오
DC1-05	30_M_BUTTON_CLICKED	행동 시나리오
DC1-06	4_H_BUTTON_CLICKED	행동 시나리오
DC1-07	1_DAY_BUTTON_CLICKED	행동 시나리오
DC1-08	1_M_BUTTON_CLICKED	행동 시나리오
DC1-09	30_M_BUTTON_CLICKED	행동 시나리오
DC1-10	1_DAY_BUTTON_CLICKED	행동 시나리오
DC1-11	1_M_BUTTON_CLICKED	행동 시나리오
DC1-12	30_M_BUTTON_CLICKED	행동 시나리오
DC1-13	4_H_BUTTON_CLICKED	행동 시나리오

DC2 — 지표 설정창

실제 경로:
ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.DISPLAY_CHART.indicator_settings

실행 정의: [chartMachine.ts](#)

설계 ID	표의 event	실행 검증
DC2-01	None	행동 시나리오
DC2-02	INDICATOR_SETTINGS_BUTTON_CLICKED	행동 시나리오
DC2-03	INDICATOR_POPUP_OUTSIDE_CLICKED	행동 시나리오

IP1 — BB 표시

실제 경로:
ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.DISPLAY_CHART.indicator_settings.opened.bollinger_bands

실행 정의: [chartMachine.ts](#)

설계 ID	표의 event	실행 검증
IP1-01	None	행동 시나리오
IP1-02	None	행동 시나리오
IP1-03	None	행동 시나리오
IP1-04	BB_DISPLAY_ON_CLICKED	행동 시나리오
IP1-05	BB_DISPLAY_OFF_CLICKED	행동 시나리오

IP2 — EMA 표시

실제 경로:
ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.DISPLAY_CHART.indicator_settings.opened.ema9

실행 정의: [chartMachine.ts](#)

설계 ID	표의 event	실행 검증
IP2-01	None	행동 시나리오
IP2-02	None	행동 시나리오
IP2-03	None	행동 시나리오
IP2-04	EMA_DISPLAY_ON_CLICKED	행동 시나리오
IP2-05	EMA_DISPLAY_OFF_CLICKED	행동 시나리오

IP3 — 거래량 표시

실제 경로: `ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.DISPLAY_CHART.indicator_settings.opened.volume`

실행 정의: `chartMachine.ts`

설계 ID	표의 event	실행 검증
IP3-01	None	행동 시나리오
IP3-02	None	행동 시나리오
IP3-03	None	행동 시나리오
IP3-04	VOLUME_DISPLAY_ON_CLICKED	행동 시나리오
IP3-05	VOLUME_DISPLAY_OFF_CLICKED	행동 시나리오

DC3 — 거래 로직 상태 표시

실제 경로: `ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.DISPLAY_CHART.active_state`

실행 정의: `chartMachine.ts`

설계 ID	표의 event	실행 검증
DC3-01	None	행동 시나리오
DC3-02	TRADING_LOGIC_STATE_CHANGED	행동 시나리오

DC4 — 차트 크기

실제 경로: `ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.DISPLAY_CHART.viewport`

실행 정의: `chartMachine.ts`

설계 ID	표의 event	실행 검증
DC4-01	None	행동 시나리오
DC4-02	FULL_SIZE_SELECTED	행동 시나리오
DC4-03	NORMAL_SIZE_SELECTED	행동 시나리오

DC5 — 선 그리기

실제 경로: `ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.DISPLAY_CHART.drawing`

실행 정의: `chartMachine.ts`

설계 ID	표의 event	실행 검증
DC5-01	None	행동 시나리오

설계 ID	표의 event	실행 검증
DC5-02	DRAWING_TOOL_CLICKED	행동 시나리오
DC5-03	USER_START_DRAWING	행동 시나리오
DC5-04	DRAWING_TOOL_CLICKED	행동 시나리오
DC5-05	USER_FINISH_DRAWING	행동 시나리오
DC5-06	DRAWING_CANCELED	행동 시나리오
DC5-07	DRAWING_TOOL_CLICKED	행동 시나리오

DC6 — 선 선택·삭제

실제 경로: ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.DISPLAY_CHART.line_selection

실행 정의: chartMachine.ts

설계 ID	표의 event	실행 검증
DC6-01	None	행동 시나리오
DC6-02	CURSOR_HOVER_ENTER	행동 시나리오
DC6-03	HIGHLIGHTED_LINE_RIGHT_CLICKED	행동 시나리오
DC6-04	CURSOR_HOVER_EXIT	행동 시나리오
DC6-05	DELETE_LINE	행동 시나리오
DC6-06	CONTEXT_MENU_OUTSIDE_CLICKED	행동 시나리오

DI1 — 투자 상태

실제 경로: ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.DISPLAY_ACCOUNT_INFO.trading_logic_status

실행 정의: accountSummaryMachine.ts

설계 ID	표의 event	실행 검증
DI1-01	None	행동 시나리오
DI1-02	TRADING_STATUS_UPDATED	행동 시나리오

DI2 — 자산 요약

실제 경로: ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.DISPLAY_ACCOUNT_INFO.asset_summary

실행 정의: accountSummaryMachine.ts

설계 ID	표의 event	실행 검증
DI2-01	None	행동 시나리오
DI2-02	ASSET_SUMMARY_UPDATED	행동 시나리오

SI — 분할 매수

실제 경로:
ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.DISPLAY_ACCOUNT_INFO.SPLIT_ORDER.scale_in.SCALE_IN_ORDER

실행 정의: [splitOrderRegions.ts](#) · [splitOrderMachine.ts](#)

설계 ID	표의 event	실행 검증
SI-01	None	행동 시나리오
SI-02	SCALE_IN_LEVEL_CHANGED	행동 시나리오

SO — 분할 매도

실제 경로:
ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.DISPLAY_ACCOUNT_INFO.SPLIT_ORDER.scale_out.SCALE_OUT_ORDER

실행 정의: [splitOrderRegions.ts](#) · [splitOrderMachine.ts](#)

설계 ID	표의 event	실행 검증
SO-01	None	행동 시나리오
SO-02	SCALE_OUT_LEVEL_CHANGED	행동 시나리오

M4 — 최근 체결·실시간 지표

실제 경로: ETIRE_UI_SYSTEM.SCREEN.MAIN_SCREEN_WRAPPER.TRADER_PANEL

실행 정의: [recentOrdersMachine.ts](#)

설계 ID	표의 event	실행 검증
M4-01	None	행동 시나리오
M4-02	BUY_ORDER_EXECUTED	행동 시나리오
M4-03	SELL_ORDER_EXECUTED	행동 시나리오
M4-04	REALTIME_INDICATOR_CLICKED	행동 시나리오
M4-05	REALTIME_INDICATOR_UPDATED	행동 시나리오
M4-06	TRADING_STATUS_UPDATED	행동 시나리오
M4-07	BUY_ORDER_EXECUTED	행동 시나리오

설계 ID	표의 event	실행 검증
M4-08	SELL_ORDER_EXECUTED	행동 시나리오
M4-09	TRADING_HISTORY_CLICKED	행동 시나리오

D1 — 수익률

실제 경로: `ETIRE_UI_SYSTEM.SCREEN.TRADING_DETAILS.ACCOUNT_DETAILS.profit_rate`

실행 정의: `tradeHistorySummaryMachine.ts`

설계 ID	표의 event	실행 검증
D1-01	None	행동 시나리오
D1-02	PROFIT_RATE_UPDATED	행동 시나리오

D2 — 매도 성과

실제 경로: `ETIRE_UI_SYSTEM.SCREEN.TRADING_DETAILS.ACCOUNT_DETAILS.sell_performance`

실행 정의: `tradeHistorySummaryMachine.ts`

설계 ID	표의 event	실행 검증
D2-01	None	행동 시나리오
D2-02	SELL_ORDER_EXECUTED	행동 시나리오

D3 — ETH 보유량

실제 경로: `ETIRE_UI_SYSTEM.SCREEN.TRADING_DETAILS.ACCOUNT_DETAILS.eth_holdings`

실행 정의: `tradeHistorySummaryMachine.ts`

설계 ID	표의 event	실행 검증
D3-01	None	행동 시나리오
D3-02	BUY_ORDER_EXECUTED	행동 시나리오
D3-03	SELL_ORDER_EXECUTED	행동 시나리오

D4 — 당일 수수료

실제 경로: `ETIRE_UI_SYSTEM.SCREEN.TRADING_DETAILS.ACCOUNT_DETAILS.daily_trading_fee`

실행 정의: `tradeHistorySummaryMachine.ts`

설계 ID	표의 event	실행 검증
D4-01	None	행동 시나리오
D4-02	DAILY_TRADING_FEE_CHANGED	행동 시나리오

TD2 — 기간 선택.결합 조회

실제 경로: ETIRE_UI_SYSTEM.SCREEN.TRADING_DETAILS.PERIOD

실행 정의: historyFilterRegions.ts · tradeHistoryMachine.ts

설계 ID	표의 event	실행 검증
TD2-01	None	행동 시나리오
TD2-02	SELECT_DISPLAY_WEEKLY_HISTORY	행동 시나리오
TD2-03	SELECT_DISPLAY_MONTHLY_HISTORY	행동 시나리오
TD2-04	SELECT_DISPLAY_ALL_HISTORY	행동 시나리오
TD2-05	SELECT_DISPLAY_TODAY_HISTORY	행동 시나리오
TD2-06	SELECT_DISPLAY_MONTHLY_HISTORY	행동 시나리오
TD2-07	SELECT_DISPLAY_ALL_HISTORY	행동 시나리오
TD2-08	SELECT_DISPLAY_TODAY_HISTORY	행동 시나리오
TD2-09	SELECT_DISPLAY_WEEKLY_HISTORY	행동 시나리오
TD2-10	SELECT_DISPLAY_ALL_HISTORY	행동 시나리오
TD2-11	SELECT_DISPLAY_TODAY_HISTORY	행동 시나리오
TD2-12	SELECT_DISPLAY_WEEKLY_HISTORY	행동 시나리오
TD2-13	SELECT_DISPLAY_MONTHLY_HISTORY	행동 시나리오

TD3 — 거래 종류 선택

실제 경로: ETIRE_UI_SYSTEM.SCREEN.TRADING_DETAILS.SIDE

실행 정의: historyFilterRegions.ts · tradeHistoryMachine.ts

설계 ID	표의 event	실행 검증
TD3-01	None	행동 시나리오
TD3-02	BUY_TRADE_HISTORY_SELECTED	행동 시나리오
TD3-03	SELL_TRADE_HISTORY_SELECTED	행동 시나리오
TD3-04	ALL_TRADE_HISTORY_SELECTED	행동 시나리오
TD3-05	SELL_TRADE_HISTORY_SELECTED	행동 시나리오
TD3-06	ALL_TRADE_HISTORY_SELECTED	행동 시나리오
TD3-07	BUY_TRADE_HISTORY_SELECTED	행동 시나리오

TD4 — CSV 내보내기

실제 경로: ETIRE_UI_SYSTEM.SCREEN.TRADING_DETAILS.CSV_EXPORT

실행 정의: csvExportRegions.ts · csvExportMachine.ts

설계 ID	표의 event	실행 검증
TD4-01	None	행동 시나리오
TD4-02	CSV_EXPORT_CLICKED	행동 시나리오
TD4-03	CLOSE_CSV_EXPORT_POPUP	행동 시나리오
TD4-04	EXPORT_CSV	행동 시나리오
TD4-05	EXPORT_CSV	행동 시나리오
TD4-06	CSV_EXPORT_SUCCEEDED	행동 시나리오
TD4-07	CSV_EXPORT_FAILED	행동 시나리오
TD4-08	CSV_EXPORT_ERROR_CONFIRMED	행동 시나리오
TD4-09	ACCEPT_CLOSE_ALL_POPUP	행동 시나리오

CR1 — CSV 경로

실제 경로: ETIRE_UI_SYSTEM.SCREEN.TRADING_DETAILS.CSV_EXPORT.editing.file_browser

실행 정의: csvExportMachine.ts

설계 ID	표의 event	실행 검증
CR1-01	None	행동 시나리오
CR1-02	SAVE_LOCATION_SELECT_CLICKED	행동 시나리오
CR1-03	SAVE_LOCATION_CONFIRMED	행동 시나리오
CR1-04	SAVE_LOCATION_CANCELED	행동 시나리오

CR2 — CSV 기간·달력

실제 경로: ETIRE_UI_SYSTEM.SCREEN.TRADING_DETAILS.CSV_EXPORT.editing.period

실행 정의: csvExportMachine.ts

설계 ID	표의 event	실행 검증
CR2-01	None	행동 시나리오
CR2-02	SELECT_CSV_WEEKLY_HISTORY	행동 시나리오
CR2-03	SELECT_CSV_MONTHLY_HISTORY	행동 시나리오
CR2-04	SELECT_CSV_DATE	행동 시나리오
CR2-05	SELECT_CSV_TODAY_HISTORY	행동 시나리오

설계 ID	표의 event	실행 검증
CR2-06	SELECT_CSV_MONTHLY_HISTORY	행동 시나리오
CR2-07	SELECT_CSV_DATE	행동 시나리오
CR2-08	SELECT_CSV_TODAY_HISTORY	행동 시나리오
CR2-09	SELECT_CSV_WEEKLY_HISTORY	행동 시나리오
CR2-10	SELECT_CSV_DATE	행동 시나리오
CR2-11	SELECT_CSV_TODAY_HISTORY	행동 시나리오
CR2-12	SELECT_CSV_WEEKLY_HISTORY	행동 시나리오
CR2-13	SELECT_CSV_MONTHLY_HISTORY	행동 시나리오
CR2-14	START_CSV_START_DATE_SELECTION	행동 시나리오
CR2-15	START_CSV_FINISH_DATE_SELECTION	행동 시나리오
CR2-16	START_DATE_SELECTED	행동 시나리오
CR2-17	START_DATE_SELECTED	행동 시나리오
CR2-18	START_DATE_CALENDAR_OUTSIDE_CLICKED	행동 시나리오
CR2-19	FINISH_DATE_SELECTED	행동 시나리오
CR2-20	FINISH_DATE_SELECTED	행동 시나리오
CR2-21	FINISH_DATE_CALENDAR_OUTSIDE_CLICKED	행동 시나리오

CR3 — CSV 파일명

실제 경로: ETIRE_UI_SYSTEM.SCREEN.TRADING_DETAILS.CSV_EXPORT.editing.file_name

실행 정의: csvExportRegions.ts · csvExportMachine.ts

설계 ID	표의 event	실행 검증
CR3-01	None	행동 시나리오
CR3-02	FILE_NAME_CLICKED	행동 시나리오
CR3-03	ENTER_KEY_TYPED	행동 시나리오
CR3-04	FILE_NAME_INPUT_FOCUS_LOST	행동 시나리오
CR3-05	ENTER_KEY_TYPED	행동 시나리오
CR3-06	FILE_NAME_INPUT_FOCUS_LOST	행동 시나리오
CR3-07	FILE_NAME_CLICKED	행동 시나리오

ES3 — 프로그램 종료

실제 경로: ETIRE_UI_SYSTEM.EXIT → UI_FINAL_STATE

실행 정의: [appExitMachine.ts](#) · [uiApplicationMachine.ts](#) · [BackendUiAdapter.ts](#)

설계 ID	표의 event	실행 검증
ES3-01	None	행동 시나리오
ES3-02	EXIT_CLICKED	행동 시나리오
ES3-03	EXIT_CLICKED	행동 시나리오
ES3-04	FORCE_SELL_EXIT_CANCELED	행동 시나리오
ES3-05	FORCE_SELL_EXIT_CONFIRMED	행동 시나리오
ES3-06	FORCE_SELL_EXIT_SUCCEEDED	행동 시나리오
ES3-07	FORCE_SELL_EXIT_FAILED	행동 시나리오
ES3-08	EXIT_CANCELED	행동 시나리오
ES3-09	EXIT_CONFIRMED	행동 시나리오